Two-Dimensional Dynamic Programming: Algorithmic Formalization, State Space Reduction, Combinatorial Proofs, and Implementation of House Robber, Unique Paths, N-Digit Numbers, and Catalan Unique BSTs

Abstract

This paper presents a formal, unified algorithmic analysis of Two-Dimensional (2D) Dynamic Programming (DP) paradigms derived from advanced computer science instruction. First, we provide an explicit reference guide detailing every asymptotic symbol, combinatorial coefficient, modular reduction, and state-space matrix notation. Second, we establish the theoretical foundations of multi-dimensional state caching, contrasting top-down memoization with bottom-up tabulation. Third, we analyze the House Robber problem, proving its Bellman recurrence $dp[i] = \max(h_i + dp[i-2], dp[i-1])$ and demonstrating space collapse from $O(N)$ to $O(1)$. Fourth, we formalize Unique Grid Paths, deriving its 2D recurrence $dp[i][j] = dp[i-1][j] + dp[i][j-1]$, establishing a combinatorial closed-form proof $\binom{N+M-2}{N-1}$, and reducing space to $O(M)$. Fifth, we model N-Digit Numbers with Digit Sum B, establishing digit-transition dynamics under leading-zero constraints and proving $O(A \cdot B)$ time and $O(B)$ space complexity. Sixth, we prove the Root Selection Theorem for Unique Binary Search Trees, deriving the Catalan Convolution recurrence $C_N = \sum_{i=1}^{N} C_{i-1}C_{N-i}$ and its closed-form $C_N = \frac{1}{N+1}\binom{2N}{N}$. Production-grade, compilable Java implementations are provided for all discussed algorithms.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Mathematical Symbols, Formal Notation, and Accessible Reference Guide

To ensure complete clarity, mathematical rigor, and broad accessibility for theoretical computer scientists, software engineers, and researchers, this section provides an explicit reference guide to every mathematical symbol, recurrence operator, asymptotic function, and combinatorial identity utilized throughout this research paper.

1.1 Asymptotic Bounds and Computational Complexity Notations

- Big-O Notation ($\mathcal{O}$ or O): Asymptotic Upper Bound
  - Formal Definition: A function $f(N) = O(g(N))$ if there exist positive constants $c > 0$ and $N_0 \geq 0$ such that $f(N) \leq c \cdot g(N)$ for all $N \geq N_0$.
  - Context in Paper: Represents the worst-case time or auxiliary memory required by an algorithm as input size scales. For example, brute-force search over all grid paths requires exponential $O(2^{N+M})$ operations, whereas 2D Dynamic Programming reduces this to polynomial $O(N \cdot M)$ operations.
- Big-Theta Notation (Θ): Asymptotically Tight Bound
  - Formal Definition: $f(N) = \Theta(g(N))$ if and only if $f(N) = O(g(N))$ and $f(N) = \Omega(g(N))$. That is, $c_1 g(N) \leq f(N) \leq c_2 g(N)$ for all $N \geq N_0$.
  - Context in Paper: Denotes exact asymptotic scaling. Tabulation of the N-Digit Numbers problem with sum B executes in exactly $\Theta(A \cdot B)$ arithmetic operations.

1.2 Combinatorics, Summation, and State Space Algebra

- Combinatorial Choice / Binomial Coefficient $\binom{n}{k}$:
  - Definition: $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ represents the number of ways to choose k un-ordered elements from a set of n elements.

  - Context in Paper: The closed-form solution for Unique Paths in an $N \times M$ grid is given by $\binom{N+M-2}{N-1}$, and the N-th Catalan number is expressed as $C_N = \frac{1}{N+1}\binom{2N}{N}$.
- Summation Operator ($\sum$) and Minimization/Maximization ($\min, \max$):
  - Definition: $\sum_{i=a}^{b} f(i)$ denotes discrete accumulation. The operators $\max(a, b)$ and $\min(a, b)$ select the extreme values between arguments.
- Modular Arithmetic Operator ($\pmod M$):
  - Definition: $a \equiv b \pmod M$ implies that $(a - b)$ is integer divisible by M. In algorithm implementations, $M = 10^9 + 7$ is used to prevent 64-bit integer overflow during exponential state counts.

TABLE 1
Mathematical Symbol and Notation Reference Table

Symbol	Name / Operation	Algorithmic Context & Domain
$O(g(N))$	Big-O Asymptotic Upper Bound	Worst-case growth rate of execution time or memory.
$\Theta(g(N))$	Big-Theta Exact Bound	Tight asymptotic complexity of dynamic programming loops.
$dp[i]$ / $dp[i][j]$	Dynamic Programming State Cell	Memoized entry storing optimal solution for subproblem (i, j).
$\binom{n}{k}$	Binomial Coefficient	Number of distinct combinations of k items chosen from n.
C_N	N-th Catalan Number	Total number of structurally unique Binary Search Trees of size N.
$\max(a, b)$	Maximization Selection	Choice operator in House Robber: $\max(\text{Rob}, \text{Skip})$.
$\sum_{d=0}^{9}$	Digit Transition Sum	Accumulation over all valid single-digit transitions $(0 \dots 9)$.
$a \pmod M$	Modular Reduction	Wraparound arithmetic with modulo $M = 10^9 + 7$.

2 Foundations of Two-Dimensional Dynamic Programming

Dynamic Programming (DP), originated by Richard Bellman in 1957, provides a formal mathematical framework for solving complex multi-stage decision problems by decomposing them into simpler overlapping subproblems.

2.1 Core Prerequisites for Dynamic Programming Applicability

An optimization, counting, or decision problem is amenable to Dynamic Programming if and only if it exhibits two foundational properties:

Definition 1 (Optimal Substructure). A problem possesses optimal substructure if an optimal solution to an instance of size (N, M) can be constructed from optimal solutions to its constituent subproblems of smaller dimensions $(< N, < M)$.

Definition 2 (Overlapping Subproblems). A problem exhibits overlapping subproblems if its recursive decomposition evaluates identical subproblems multiple times across distinct branches of the call tree.

2.2 Architectural Approaches: Top-Down Memoization vs. Bottom-Up Tabulation

Dynamic Programming algorithms are realized through two primary paradigms:

1) Top-Down Dynamic Programming (Memoization): Recursion is maintained following the problem's natural top-down decomposition, but evaluated states are stored in a lookup table (e.g., hash map or 2D array). Upon encountering a previously solved state (i, j), the stored result is returned in $O(1)$ time, pruning redundant call subtrees.
2) Bottom-Up Dynamic Programming (Tabulation): Recursion is eliminated entirely. An n-dimensional state table is initialized with base cases, and populated iteratively in topologically sorted order from smallest subproblems to the target problem state.

Theorem 3 (Complexity Collapse via Caching). Let $T_{rec}(N)$ be the time complexity of an un-memoized tree recursion with branching factor b and depth d, yielding $T_{rec}(N) = O(b^d)$. If the total number of distinct state tuples is S and each state transition requires $O(k)$ work, Top-Down Memoization and Bottom-Up Tabulation collapse the overall time complexity to $T_{dp}(N) = \Theta(S \cdot k)$.

3 1D Non-Adjacent Subsequence Optimization: House Robber Problem

To establish the bridge between one-dimensional state transitions and higher-dimensional DP grids, we first analyze the Maximum Sum without Adjacent Elements problem (popularly known as the House Robber problem).

3.1 Formal Problem Definition

Given an array $H = [h_0, h_1, h_2, \dots, h_{N-1}]$ of non-negative integers where h_i represents the value of the i-th house, select a subset of indices $I \subseteq \{0, 1, \dots, N-1\}$ such that no two selected indices are adjacent (i.e., $|i - j| > 1$ for all $i, j \in I, i \neq j$) and the sum $\sum_{i \in I} h_i$ is maximized.

3.2 Mathematical Recurrence and State Space Derivation

Let $dp[i]$ denote the maximum value obtainable from the prefix subarray $H[0 \dots i]$. At index i, two mutually exclusive actions are available:

1) Action 1 (Rob House i): We add h_i to the maximum total obtainable from non-adjacent houses up to index $i-2$, yielding $h_i + dp[i-2]$.
2) Action 2 (Skip House i): We exclude house i, maintaining the maximum total obtainable up to index $i-1$, yielding $dp[i-1]$.

Taking the maximum across both choices yields the fundamental Bellman recurrence:

$$dp[i] = \max\Big(h_i + dp[i-2],\, dp[i-1]\Big) \quad \text{for } i \geq 2 \tag{1}$$

3.2.1 Base Cases

- $dp[0] = h_0$ (with only one house, robbing it is optimal).
- $dp[1] = \max(h_0, h_1)$ (with two houses, pick the wealthier one).

3.3 Space Optimization: Collapse from $O(N)$ to $O(1)$

Notice that computing $dp[i]$ requires only $dp[i-1]$ and $dp[i-2]$. Consequently, storing the entire array of size N is unnecessary. We can maintain two scalar state variables, prev2 $\equiv dp[i-2]$ and prev1 $\equiv dp[i-1]$, updating them in linear time:

$$\text{curr} = \max(h_i + \text{prev2},\, \text{prev1})$$

This space optimization reduces auxiliary space complexity from $O(N)$ to $O(1)$.

3.4 Production-Grade Java Implementation

```
public class HouseRobber {
    /**
     * Solves House Robber using Tabulation with O(1) Space Complexity.
     * @param h Array representing wealth at each house.
     * @return Maximum total wealth obtainable without robbing adjacent houses.
     */
    public static int robOptimized(int[] h) {
        if (h == null || h.length == 0) return 0;
        if (h.length == 1) return h[0];

        int prev2 = h[0];
        int prev1 = Math.max(h[0], h[1]);

        for (int i = 2; i < h.length; i++) {
            int curr = Math.max(h[i] + prev2, prev1);
            prev2 = prev1;
            prev1 = curr;
        }

        return prev1;
    }
}
```

4 Grid-Based Spatial Optimization: Unique Paths in 2D Grids

4.1 Formal Problem Definition

Consider an $N \times M$ grid where a robot starts at the top-left cell $(0,0)$ and aims to reach the bottom-right cell $(N-1, M-1)$. The robot can only move either Right (from (i,j) to $(i, j+1)$) or Down (from (i,j) to $(i+1, j)$). We determine the total number of distinct paths from $(0,0)$ to $(N-1, M-1)$.

4.2 Combinatorial Closed-Form Proof

Theorem 4 (Grid Path Combinatorics). The total number of unique paths in an $N \times M$ grid from $(0, 0)$ to $(N-1, M-1)$ moving only Right and Down is exactly:

$$P(N, M) = \binom{(N-1)+(M-1)}{N-1} = \frac{(N+M-2)!}{(N-1)!\,(M-1)!} \tag{2}$$

Proof. To travel from $(0, 0)$ to $(N-1, M-1)$, any valid path must make exactly $(N-1)$ Down moves and $(M-1)$ Right moves. The total number of moves in any path is $(N-1)+(M-1) = N+M-2$. A path is uniquely determined by choosing which $(N-1)$ positions out of the total $(N+M-2)$ steps are Down moves. The number of ways to choose these steps is given by the binomial coefficient $\binom{N+M-2}{N-1}$. □

4.3 2D Dynamic Programming Formulation

Let $dp[i][j]$ denote the total number of unique paths from cell $(0, 0)$ to cell (i, j).

4.3.1 Recurrence Relation

Since cell (i, j) can only be reached from the cell directly above it $(i-1, j)$ or the cell directly to its left $(i, j-1)$, the total ways to reach (i, j) is the sum of ways to reach those predecessors:

$$dp[i][j] = dp[i-1][j] + dp[i][j-1] \quad \text{for } 1 \le i < N,\ 1 \le j < M \tag{3}$$

4.3.2 Boundary / Base Conditions

- $dp[i][0] = 1$ for all $0 \le i < N$ (cells in the first column have only 1 path: moving strictly Down).
- $dp[0][j] = 1$ for all $0 \le j < M$ (cells in the first row have only 1 path: moving strictly Right).

4.4 Space Reduction: From 2D Matrix $O(N \cdot M)$ to 1D Buffer $O(M)$

In the 2D tabulation, updating row i requires only the values from row $i-1$ (directly above) and the current row's left neighbour $dp[i][j-1]$. Thus, we can maintain a single 1D array 'dp' of size M. As we iterate left-to-right through row i:

$$dp[j] = dp[j] + dp[j-1]$$

where 'dp[j]' on the right side holds the value from the previous row $i-1$, and 'dp[j-1]' holds the newly computed value for the current row.

4.5 Production-Grade Java Implementation

```
import java.util.Arrays;

public class UniquePaths {
    /**
     * Computes total unique grid paths using 1D Space-Optimized DP.
     * @param n Number of rows in grid.
     * @param m Number of columns in grid.
     * @return Total distinct paths from (0,0) to (n-1, m-1).
     */
    public static int uniquePaths(int n, int m) {
        int[] dp = new int[m];
        Arrays.fill(dp, 1); // Base case for first row

        for (int i = 1; i < n; i++) {
            for (int j = 1; j < m; j++) {
                dp[j] += dp[j - 1];
            }
        }
        return dp[m - 1];
    }
}
```

5 Digit-Constrained State Spaces: N-Digit Numbers with Digit Sum B

5.1 Formal Problem Definition

Find the total count of positive A-digit numbers whose individual digits sum up to B. A valid A-digit number cannot contain a leading zero (i.e., its first digit must be between 1 and 9). Because the count can be extremely large, return the result modulo $M = 10^9 + 7$.

5.2 State Representation and Subproblem Decomposition

Let $dp[i][s]$ represent the number of valid i-digit suffixes whose digits sum up to s.

5.2.1 First Digit Constraints (Position $i = 1$)

For the first digit of an A-digit number, the digit d can take values from $1 \leq d \leq 9$. Thus, for a 1-digit number ($i = 1$):

$$dp[1][s] = \begin{cases} 1 & \text{if } 1 \leq s \leq \min(9, B) \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

5.2.2 General Digit Transitions ($2 \leq i \leq A$)

For subsequent digit positions ($i \geq 2$), the appended digit d can take any value from $0 \leq d \leq 9$, provided that $s - d \geq 0$. The state transition recurrence is given by:

$$dp[i][s] = \sum_{d=0}^{\min(9,s)} dp[i-1][s-d] \pmod{10^9 + 7} \tag{5}$$

5.3 Complexity Analysis and Proof

Theorem 5 (Complexity of N-Digit Sum Tabulation). The tabulation algorithm for counting A-digit numbers with digit sum B runs in $\Theta(A \cdot B)$ time and $O(B)$ auxiliary space.

Proof. The state table consists of $A \times (B + 1)$ entries. Filling each cell (i, s) requires iterating over $d \in [0, 9]$, performing at most 10 operations (a constant $O(1)$ factor). Thus, total execution time is bounded by $\Theta(A \cdot B \cdot 10) = \Theta(A \cdot B)$. Since row i depends exclusively on row $i - 1$, we store only two 1D arrays of size $B + 1$, consuming $O(B)$ auxiliary memory. □

5.4 Production-Grade Java Implementation

```
public class NDigitNumbersWithSum {
    private static final int MOD = 1000000007;

    /**
     * Computes the number of A-digit numbers with digit sum B.
     * @param A Number of digits.
     * @param B Target sum of digits.
     * @return Total valid numbers modulo 10^9 + 7.
     */
    public static int solve(int A, int B) {
        if (A <= 0 || B <= 0) return 0;

        int[] prev = new int[B + 1];

        // Base case: First digit (1 to 9)
        for (int d = 1; d <= 9; d++) {
            if (d <= B) {
                prev[d] = 1;
            }
        }

        // Fill DP table for digits 2 to A
        for (int i = 2; i <= A; i++) {
            int[] curr = new int[B + 1];
            for (int s = 1; s <= B; s++) {
                long sumWays = 0;
                for (int d = 0; d <= 9; d++) {
                    if (s - d >= 0) {
                        sumWays = (sumWays + prev[s - d]) % MOD;
                    }
                }
                curr[s] = (int) sumWays;
            }
            prev = curr;
        }
```

```
        return prev[B];
    }
}
```

6 Structural Recursive Counting: Catalan Numbers and Unique BSTs

6.1 Formal Problem Definition

Given an integer N, count the total number of structurally unique Binary Search Trees (BSTs) that can be constructed using N distinct key values (e.g., $1, 2, \ldots, N$).

6.2 Root Selection Theorem and Structural Decomposition

Theorem 6 (Recursive BST Decomposition). Let C_N be the number of unique BSTs with N nodes. Then C_N satisfies the Catalan convolution recurrence:

$$C_N = \sum_{i=1}^{N} C_{i-1} \cdot C_{N-i} \quad \text{for } N \geq 1, \quad \text{with } C_0 = 1 \tag{6}$$

Proof. Any key $i \in \{1, 2, \ldots, N\}$ can be selected as the root of the BST.

1) Keys strictly smaller than i ($\{1, \ldots, i-1\}$) must be placed in the left subtree. There are $i-1$ such keys, which can form C_{i-1} unique left subtrees.
2) Keys strictly greater than i ($\{i+1, \ldots, N\}$) must be placed in the right subtree. There are $N-i$ such keys, which can form C_{N-i} unique right subtrees.

By the Fundamental Counting Principle, for a fixed root i, the number of unique BSTs is $C_{i-1} \times C_{N-i}$. Summing across all possible root choices $i \in [1, N]$ yields Eq. (6). □

6.3 Catalan Sequence Values and Closed-Form Formula

The first few terms of the Catalan sequence are:

$$C_0 = 1, \quad C_1 = 1, \quad C_2 = 2, \quad C_3 = 5, \quad C_4 = 14, \quad C_5 = 42, \quad C_6 = 132$$

Using generating functions, the closed-form analytic expression for the N-th Catalan number is:

$$C_N = \frac{1}{N+1}\binom{2N}{N} = \frac{(2N)!}{(N+1)!\,N!} \tag{7}$$

6.4 Production-Grade Java Implementation

```
public class UniqueBSTs {
    /**
     * Solves Unique BST count using O(N^2) Tabulation.
     * @param N Number of distinct nodes.
     * @return N-th Catalan number (total unique BSTs).
     */
    public static int numTrees(int N) {
        if (N <= 1) return 1;

        int[] dp = new int[N + 1];
        dp[0] = 1;
        dp[1] = 1;

        for (int n = 2; n <= N; n++) {
            for (int i = 1; i <= n; i++) {
                dp[n] += dp[i - 1] * dp[n - i];
            }
        }
        return dp[N];
    }
}
```

7 Comparative Algorithmic Taxonomy and Complexity Synthesis

To summarize the theoretical performance and state characteristics of the 2D Dynamic Programming problems analyzed in this paper, Table 2 synthesizes their state dimensions, time complexities, space bounds, and architectural characteristics.

TABLE 2
Comprehensive Algorithmic Comparison of Analyzed DP Problems

Problem Domain	State Dim.	Recursive Time	Tabulated Time	Naive Space	Optimized Space
House Robber	1D ($dp[i]$)	$O(2^N)$	$O(N)$	$O(N)$	$O(1)$
Unique Grid Paths	2D ($dp[i][j]$)	$O(2^{N+M})$	$O(N \cdot M)$	$O(N \cdot M)$	$O(M)$ / $O(1)$ Math
N-Digit Digit Sum	2D ($dp[i][s]$)	$O(10^A)$	$O(A \cdot B)$	$O(A \cdot B)$	$O(B)$
Unique BSTs (Catalan)	1D ($dp[N]$)	$O(3^N)$	$O(N^2)$	$O(N)$	$O(1)$ Math

8 Conclusion

This research paper presented a formal, unified analysis of two-dimensional dynamic programming techniques. We demonstrated how state caching and structural recurrence relations transform intractable exponential search spaces into highly efficient polynomial or linear time algorithms. By establishing formal proofs for optimal substructure, space optimizations, and combinatorial equivalents, this work serves as an authoritative guide for advanced algorithm design.